CS456 Design and Analysis of Algorithms
Project 5: Load Balancing

Deepshan Adhikari (800846035)
deepadh@siue.edu

1 Introduction

1.1 Problem Statement

The Load Balancing problem assigns a set of n jobs, each with processing time t_j, to a set of m machines so that the maximum load on any machine is minimized. This maximum load is called the **makespan**, denoted by T. Finding an assignment of minimum makespan is **NP-hard** [1], so approximation algorithms are used to trade optimality for tractability.

1.2 Approach

Three algorithms are examined: List Scheduling, Sorted List Scheduling, and Local Search. All three are run on the same job set per trial. The experiment uses 10 machines and 200 jobs with processing times drawn uniformly at random from the integers 1 to 25. The experiment is repeated for 10 independent trials, and the highest, average, and lowest makespan is recorded for each algorithm, along with average running times.

1.3 Algorithms at a Glance

- **List Scheduling:** A greedy algorithm that processes jobs in the given order and always assigns the next job to the machine with the smallest current load. It runs in a single pass and guarantees a makespan no more than twice the optimal, making it a **2-approximation**. Its weakness is that if a large job arrives near the end, every machine is already loaded with smaller jobs, so placing it anywhere pushes that machine far above the others and inflates the makespan.

- **Sorted List Scheduling:** The same greedy rule as List Scheduling, but jobs are first sorted in descending order so the largest jobs are placed while machines are still empty. This fixes the late-big-job problem and tightens the guarantee to $\frac{3}{2}$ of the optimal, making it a **3/2-approximation**. It costs a little extra time for sorting but gives better and more consistent results.

- **Local Search:** A heuristic that starts from a random assignment and repeatedly swaps two jobs between two machines whenever the swap reduces the maximum load of those two machines, restarting after each improvement and stopping when no improving swap remains. In practice, it often gets close to optimal, but it has **no constant-factor approximation guarantee** and is much slower than the other two.

1.4 Theoretical Time Complexity

Algorithm	Time Complexity	Approximation Factor
List Scheduling	$O(nm)$	$T \leq 2T^*$
Sorted List Scheduling	$O(n \log n + nm)$	$T \leq 1.5T^*$
Local Search	$O(n^2)$ per pass; total $O(P^2n^2)$ worst case	No proven bound

Table 1: Theoretical time complexity and approximation factors.

List Scheduling runs in $O(nm)$ because each of the n jobs requires a linear scan over m machine loads to find the minimum. **Sorted List Scheduling** adds an $O(n \log n)$ sort before the same loop, giving $O(n \log n + nm)$. **Local Search** has no polynomial bound in n and m alone because the number of improving swaps depends on processing times. One complete pass examines all machine pairs and all job pairs and costs $O(n^2)$. Each accepted swap strictly reduces the sum of squared machine loads, which is at most P^2, so the number of passes is at most $O(P^2)$. The total worst-case time is therefore $O(P^2n^2)$, a loose pseudo-polynomial bound, where P is the total processing time.

1.5 Implementation Details

1.5.1 Programming Language and Its Effect

The code is written in C++17. C++ is a compiled language with low runtime overhead, so all three algorithms run very quickly and the measured times reflect the algorithms themselves rather than the language. This keeps the running-time comparison meaningful. Because the times are small, they are sensitive to noise, so `std::chrono::steady_clock` was used and results were averaged over 10 trials.

1.5.2 Experimental Setup

The setup used 10 machines and 200 jobs. Processing times were drawn uniformly at random from the integers 1 to 25. For each trial, the lower bound on the optimal makespan was computed as the sum of all job times divided by the number of machines, rounded up to the nearest integer. All three algorithms were then run on the same job set, and each makespan was compared to the lower bound. The entire process was repeated 10 times. For each algorithm, the highest, lowest, and average makespan were recorded, along with the running time per run. Times are reported in milliseconds.

List Scheduling processes each job in order and assigns it to the machine with the smallest current load. Sorted List Scheduling sorts jobs in descending order of processing time and then applies List Scheduling. Local Search begins by randomly partitioning jobs among machines. It then searches for an improving swap, where job x on machine a is exchanged with job y on machine b if $\max(L_a - x + y,\ L_b - y + x) < \max(L_a, L_b)$.

1.5.3 Problems Encountered

No significant problems were encountered. The code compiled and ran without errors, and all three algorithms produced results consistent with the expected behavior. The only minor issue handled was keeping Local Search random draws independent of job generation by using a separate random generator.

2 Proof: List Scheduling is a 2-Approximation

Claim. Let T be the makespan produced by List Scheduling and T^* the optimal makespan. Then $T \leq 2T^*$.

Notation. Job j has processing time t_j, and L_k is the final load of machine k. Let machine i be a machine whose load equals the makespan, so $L_i = T$.

Lemma 1 (Average load bound). The optimal makespan is at least the average load:

$$T^* \geq \frac{1}{m} \sum_{j=1}^{n} t_j$$

Proof. The total work $P = \sum_j t_j$ must be spread over m machines, and the busiest machine carries at least the average load.

Lemma 2 (Largest job bound). The optimal makespan is at least the largest job:

$$T^* \geq \max_j t_j$$

Proof. Some machine must run the longest job, so the makespan is at least that job's length.

Proof of the claim. Let j be the last job assigned to machine i. Just before job j was assigned, machine i had load $L_i - t_j$. List Scheduling assigns each job to the machine with the smallest current load, so at that moment the load of machine i was no larger than the load of every other machine k. Loads only increase over time, so the final load of every machine k satisfies $L_k \geq L_i - t_j$. Summing over all m machines gives:

$$m(L_i - t_j) \leq \sum_k L_k = \sum_j t_j$$

Dividing by m and applying Lemma 1 yields:

$$L_i - t_j \leq \frac{1}{m} \sum_{j=1}^{n} t_j \leq T^*$$

The makespan equals the load before job j plus the length of job j. Applying the inequality above to the first term and Lemma 2 to the second term gives:

$$T = L_i = (L_i - t_j) + t_j \leq T^* + T^* = 2T^*$$

This proves that $T \leq 2T^*$.

Tightness. The bound is nearly tight. If $m(m-1)$ unit jobs arrive first and one job of length m arrives last, List Scheduling gives $T = 2m - 1$ while the optimum is m. The ratio approaches 2 as m grows. This is exactly the weakness Sorted List Scheduling avoids.

3 Results

3.1 Per-Trial Results

Trial	Lower Bound	List	Sorted	Local
1	252.0	265	252	252
2	254.0	270	254	254
3	268.0	276	268	268
4	261.0	269	261	261
5	267.0	276	267	267
6	231.0	243	231	231
7	271.0	279	271	271
8	271.0	282	271	271
9	252.0	260	252	252
10	256.0	267	256	256

Table 2: Per-trial makespans and lower bound. Values are in processing-time units.

3.2 Makespan Summary

Statistic	Lower Bound	List	Sorted	Local
Highest	271	282	271	271
Average	258.3	268.7	258.3	258.3
Lowest	231	243	231	231

Table 3: Makespan results across 10 trials. Values are in processing-time units.

3.3 Observed Makespan / Lower Bound Ratios

Algorithm	Worst	Average	Guarantee
List Scheduling	1.0630	1.0406	2
Sorted List Scheduling	1.0000	1.0000	3/2
Local Search	1.0000	1.0000	none

Table 4: Ratios are dimensionless (makespan divided by lower bound).

3.4 Average Running Times

Algorithm	Average Running Time (ms)
List Scheduling	0.0117
Sorted List Scheduling	0.0403
Local Search	8.1852

Table 5: Average running times in milliseconds.

4 Discussion of Results

Sorted List Scheduling and Local Search produced identical makespans in all 10 trials, and both matched the lower bound, while List Scheduling was consistently worse.

Closeness of the algorithms. The average makespan of both Sorted List Scheduling and Local Search was 258.3, which is the average lower bound rounded up in each trial. For example, in trial 1 the lower bound was 252.0 and both algorithms achieved 252. In trial 3 the lower bound was 268.0 and both achieved 268. Because job times are integers, no schedule can have a makespan below the rounded-up lower bound, so the optimal makespan T^* is at least that value. Both algorithms reached it, which means they found an optimal schedule in all 10 trials. List Scheduling was consistently higher. Its average makespan was 268.7, about 10.4 above the average lower bound. The worst gap was 12 (trial 3: lower bound 268, List 276) and the smallest gap was 8 (trial 9: lower bound 252, List 260). The gap comes from the last jobs placed: with job sizes up to 25 arriving in random order, the final job can land on a machine that is already near the maximum load.

Approximation factors. The ratios in the ratio table are computed against the lower bound, which is at most T^*, so the true ratios (against T^*) are equal to or smaller than the reported values. All three algorithms satisfied their guarantees in every trial, and by a wide margin. List Scheduling's worst ratio was 1.0630, far below its $2\times$ guarantee. Sorted List Scheduling's worst ratio was exactly 1.0000, far below its $1.5\times$ guarantee. The guarantees are bounds for the worst case over adversarial inputs, such as many small jobs followed by one large job (Section 2), and random inputs are far from that case.

Local Search guarantee. The standard 2-approximation argument for local search applies to moves of a single job. The version implemented here only swaps two jobs between machines. This preserves the total number of jobs on each machine during the search, but the count is not fixed at any particular value because the initial partition is random. No approximation factor is claimed for this swap-only variant. Empirically it reached the optimum in all 10 trials, matching Sorted List Scheduling exactly.

Limits of the experiment. These results come from a single regime ($n/m = 20$) with a narrow range of job sizes and 10 trials from one seed. List Scheduling would be expected to perform relatively worse with fewer jobs per machine or when a large job arrives last, but those cases were not tested.

5 Discussion of Running Times

Local Search was several hundred times slower than the two greedy algorithms, both of which finish well under a millisecond.

Algorithm	**Theoretical Complexity**	**Observed Average (ms)**
List Scheduling	$O(n \cdot m)$	0.0117
Sorted List Scheduling	$O(n \log n + n \cdot m)$	0.0403
Local Search	$O(n^2)$ per pass; total $O(P^2 \cdot n^2)$	8.1852

Table 6: Theoretical vs. observed running times.

List Scheduling compared with Sorted List Scheduling. Sorted List Scheduling was about 3.4 times slower than List Scheduling (0.0403 ms vs. 0.0117 ms). This is consistent with the extra $O(n \log n)$ sort needed to order 200 jobs before scheduling. Both algorithms are extremely fast in absolute terms, and the difference in their running times is negligible for this problem size.

Local Search. It averaged 8.1852 ms, about 203 times the Sorted List Scheduling average and about 700 times the List Scheduling average. This agrees with the complexity analysis. Each pass over all machine pairs and job pairs tests roughly $n^2/2 = 20{,}000$ swaps, and every improving swap restarts the scan from the beginning. The total work is therefore roughly the number of improving swaps multiplied by the average cost of a partial scan. Because both greedy algorithms already reached the lower bound, Local Search does not offer a quality advantage here to justify its much higher cost.

Trade-off. On this data, Sorted List Scheduling is the best practical choice. It reached the same makespan as Local Search, has a proven 3/2-approximation guarantee, and is only about 3.4 times slower than unsorted List Scheduling, which is itself under 0.02 ms per trial. Local Search is not worth its extra cost for this problem size.

Timing note. Each algorithm is timed only once per trial. Because the times are so small, a single noisy measurement can shift the average noticeably. Running each algorithm multiple times per trial and averaging would give more stable numbers, but the current results are sufficient to show the relative order of the three algorithms.

6 Conclusion

The program correctly implements the three required load balancing algorithms and satisfies the experimental setup: 10 machines, 200 jobs per trial, processing times from 1 to 25, 10 trials, makespan summaries, and average running times.

The proof in Section 2 establishes that List Scheduling is a 2-approximation, and the tightness example shows the bound is nearly tight in the worst case. Sorted List Scheduling and Local Search produced identical makespans in all 10 trials and both reached the lower bound, which is the best possible for this data. List Scheduling performed worse but stayed well within its 2-approximation guarantee.

The results do not establish the exact optimum or any approximation guarantee for Local Search. They only show that the algorithms behave as expected on this small random workload. Sorted List Scheduling is the best practical choice here: it matches Local Search in quality, has a proven guarantee, and is only slightly slower than unsorted List Scheduling.

References

[1] Kleinberg, Jon & Tardos, Éva. (2006). *Algorithm Design*. Pearson/Addison-Wesley.

A Source Code

```
// CS456 Design and Analysis of Algorithms
// Project 5 - Load Balancing

// This program implements three algorithms:
//   1. List Scheduling          - O(n*m) linear scan, 2-approximation
//   2. Sorted List Scheduling   - O(n log n + n*m), 3/2-approximation
//   3. Local Search             - Iterative improving-swap heuristic

// The experiment uses 10 machines and 200 jobs with random processing
// times between 1 and 25. For each trial, the lower bound on the
// makespan is computed as ceil(total processing time / machines).
// All three algorithms run on the same job set. The whole experiment
// is repeated 10 times, tracking the high, low, and average makespan
// for each algorithm as well as the running times.

#include <iostream>  // prints experiment results
#include <vector>    // stores job lists, machine assignments, and loads
```

```cpp
#include <algorithm> // provides sort, max_element, and min_element
#include <random>    // provides the random number generator used to build jobs
#include <chrono>    // provides a monotonic timer used to measure runtime
#include <numeric>   // provides accumulate, used to sum job times for the lower bound
#include <iomanip>   // provides setw and setprecision for aligned output

using namespace std;
using namespace chrono;

// Problem constants. Ten machines and two hundred jobs, as required.
const int M = 10;       // number of machines
const int N = 200;      // number of jobs per trial
const int TRIALS = 10; // number of independent repetitions of the experiment

// Algorithm 1: List Scheduling
// Assign each job in the given order to the machine with the smallest
// current load. Returns the makespan, the maximum load on any machine.
// This is the classic 2-approximation greedy algorithm.
// Note: the job sets J(i) from the pseudocode are not stored because
// only the loads matter for computing the makespan.
int listScheduling(const vector<int> &jobs)
{
    vector<int> load(M, 0); // load[i] is the total time on machine i

    for (int jobTime : jobs)
    {
        // Find the machine with the smallest current load.
        int bestMachine = 0;
        for (int i = 1; i < M; i++)
        {
            if (load[i] < load[bestMachine])
                bestMachine = i;
        }
        load[bestMachine] += jobTime;
    }

    // The makespan is the largest load across all machines.
```

```
55      return *max_element(load.begin(), load.end());
56  }
57
58  // Algorithm 2: Sorted List Scheduling
59  // Sort jobs in descending order first, then apply List Scheduling.
60  // Placing the largest jobs first gives the LPT rule. Its standard
        bound
61  // is stronger than 3/2 for m >= 2; 3/2 is a valid, looser guarantee
        .
62  int sortedListScheduling(vector<int> jobs)
63  {
64      sort(jobs.rbegin(), jobs.rend()); // descending order
65      return listScheduling(jobs);
66  }
67
68  // Algorithm 3: Local Search
69  // Randomly partition the jobs among the machines. Then repeatedly
        look
70  // for a swap of two jobs between two machines that lowers the
        maximum
71  // load of those two machines. When an improving swap is found,
        perform
72  // it immediately and restart the search from the beginning. Stop
        when
73  // a full pass finds no improving swap.
74  int localSearch(const vector<int> &jobs, mt19937 &rng)
75  {
76      vector<vector<int>> machine(M); // jobs assigned to each machine
77      vector<int> load(M, 0);         // total load on each machine
78
79      // Random initial partition: place each job on a random machine.
80      uniform_int_distribution<int> machineDist(0, M - 1);
81      for (int jobTime : jobs)
82      {
83          int i = machineDist(rng);
84          machine[i].push_back(jobTime);
85          load[i] += jobTime;
86      }
87
88      bool improved = true;
89      while (improved)
90      {
91          improved = false;
92
93          // Try every pair of machines.
94          for (int a = 0; a < M; a++)
95          {
```

```
96              for (int b = a + 1; b < M; b++)
97              {
98                  int oldMax = max(load[a], load[b]);
99
100                 // Try every pair of jobs between the two machines.
101                 for (size_t x = 0; x < machine[a].size(); x++)
102                 {
103                     for (size_t y = 0; y < machine[b].size(); y++)
104                     {
105                         int jobA = machine[a][x];
106                         int jobB = machine[b][y];
107                         int newA = load[a] - jobA + jobB;
108                         int newB = load[b] - jobB + jobA;
109
110                         // Accept the swap only if it lowers the max
                                load
111                         // of the two machines involved.
112                         if (max(newA, newB) < oldMax)
113                         {
114                             machine[a][x] = jobB;
115                             machine[b][y] = jobA;
116                             load[a] = newA;
117                             load[b] = newB;
118                             improved = true;
119
120                             // Restart the search from the beginning
121                             // after performing an improving swap.
                                   The
122                             // goto breaks out of four nested loops
                                   at once.
123                             goto restart;
124                         }
125                     }
126                 }
127             }
128         }
129     restart:; // label for the goto above; restarts the while loop
130     }
131
132     return *max_element(load.begin(), load.end());
133 }
134
135 // Statistics helper
136 // Holds the highest, lowest, and average value of a set of results.
137 struct Stats
138 {
139     int high;
```

```
140     int low;
141     double avg;
142 };
143 
144 // Computes the high, low, and average of a vector of integers.
145 Stats summarize(const vector<int> &values)
146 {
147     int high = *max_element(values.begin(), values.end());
148     int low = *min_element(values.begin(), values.end());
149     double avg = accumulate(values.begin(), values.end(), 0.0) /
            values.size();
150     return {high, low, avg};
151 }
152 
153 int main()
154 {
155     mt19937 jobsRng(42);
156     mt19937 localSearchRng(7);
157 
158     uniform_int_distribution<int> dist(1, 25);
159 
160     vector<int> lowerBoundList;
161     vector<int> listResults;
162     vector<int> sortedResults;
163     vector<int> localResults;
164 
165     vector<double> listTime;
166     vector<double> sortedTime;
167     vector<double> localTime;
168 
169     for (int trial = 0; trial < TRIALS; trial++)
170     {
171         vector<int> jobs(N);
172         for (int &t : jobs)
173             t = dist(jobsRng);
174 
175         int total = accumulate(jobs.begin(), jobs.end(), 0);
176         lowerBoundList.push_back((total + M - 1) / M);
177 
178         auto startTime = steady_clock::now();
179         int result1 = listScheduling(jobs);
180         auto endTime = steady_clock::now();
181         listResults.push_back(result1);
182         listTime.push_back(duration_cast<microseconds>(endTime -
            startTime).count() / 1000.0);
183 
184         startTime = steady_clock::now();
```

```
185         int result2 = sortedListScheduling(jobs);
186         endTime = steady_clock::now();
187         sortedResults.push_back(result2);
188         sortedTime.push_back(duration_cast<microseconds>(endTime -
                startTime).count() / 1000.0);
189
190         startTime = steady_clock::now();
191         int result3 = localSearch(jobs, localSearchRng);
192         endTime = steady_clock::now();
193         localResults.push_back(result3);
194         localTime.push_back(duration_cast<microseconds>(endTime -
                startTime).count() / 1000.0);
195     }
196
197     // Per-trial table
198     cout << "\nPer-Trial Results\n";
199     cout << "
           ------------------------------------------------------------\
           n";
200     cout << left << setw(8) << "Trial"
201          << setw(14) << "Lower Bound"
202          << setw(10) << "List"
203          << setw(10) << "Sorted"
204          << setw(10) << "Local" << "\n";
205     cout << "
           ------------------------------------------------------------\
           n";
206
207     for (int i = 0; i < TRIALS; i++)
208     {
209         cout << left << setw(8) << (i + 1)
210              << setw(14) << fixed << setprecision(1) << (double)
                    lowerBoundList[i]
211              << setw(10) << listResults[i]
212              << setw(10) << sortedResults[i]
213              << setw(10) << localResults[i] << "\n";
214     }
215     cout << "
           ------------------------------------------------------------\
           n";
216
217     // Makespan summary
218     Stats lb = summarize(lowerBoundList);
219     Stats ls = summarize(listResults);
220     Stats sl = summarize(sortedResults);
221     Stats lo = summarize(localResults);
222
```

```
223     cout << "\nMakespan Summary\n";
224     cout << "
            ------------------------------------------------------------\
        n";
225     cout << left << setw(10) << ""
226          << setw(14) << "Lower Bound"
227          << setw(10) << "List"
228          << setw(10) << "Sorted"
229          << setw(10) << "Local" << "\n";
230     cout << left << setw(10) << "Highest"
231          << setw(14) << lb.high
232          << setw(10) << ls.high
233          << setw(10) << sl.high
234          << setw(10) << lo.high << "\n";
235     cout << left << setw(10) << "Average"
236          << setw(14) << fixed << setprecision(1) << lb.avg
237          << setw(10) << ls.avg
238          << setw(10) << sl.avg
239          << setw(10) << lo.avg << "\n";
240     cout << left << setw(10) << "Lowest"
241          << setw(14) << lb.low
242          << setw(10) << ls.low
243          << setw(10) << sl.low
244          << setw(10) << lo.low << "\n";
245     cout << "
            ------------------------------------------------------------\
        n";
246
247     // Approximation ratio table
248     double worstList = 0, avgList = 0;
249     double worstSorted = 0, avgSorted = 0;
250     double worstLocal = 0, avgLocal = 0;
251
252     for (int i = 0; i < TRIALS; i++)
253     {
254         double lbd = (double)lowerBoundList[i];
255         double rList = listResults[i] / lbd;
256         double rSorted = sortedResults[i] / lbd;
257         double rLocal = localResults[i] / lbd;
258
259         worstList = max(worstList, rList);
260         worstSorted = max(worstSorted, rSorted);
261         worstLocal = max(worstLocal, rLocal);
262
263         avgList += rList;
264         avgSorted += rSorted;
265         avgLocal += rLocal;
```

```
266     }
267     avgList /= TRIALS;
268     avgSorted /= TRIALS;
269     avgLocal /= TRIALS;
270 
271     cout << "\nObserved Makespan / Lower Bound\n";
272     cout << "
           ------------------------------------------------------------\
           n";
273     cout << left << setw(14) << "Algorithm"
274          << setw(14) << "Worst"
275          << setw(14) << "Average"
276          << setw(14) << "Guarantee" << "\n";
277     cout << left << setw(14) << "List"
278          << setw(14) << fixed << setprecision(4) << worstList
279          << setw(14) << avgList
280          << setw(14) << "2" << "\n";
281     cout << left << setw(14) << "Sorted"
282          << setw(14) << worstSorted
283          << setw(14) << avgSorted
284          << setw(14) << "3/2" << "\n";
285     cout << left << setw(14) << "Local"
286          << setw(14) << worstLocal
287          << setw(14) << avgLocal
288          << setw(14) << "none" << "\n";
289     cout << "
           ------------------------------------------------------------\
           n";
290 
291     // Running time summary
292     auto averageTime = [](const vector<double> &values)
293     {
294         return accumulate(values.begin(), values.end(), 0.0) /
              values.size();
295     };
296 
297     cout << "\nAverage Running Times (in milliseconds)\n";
298     cout << "
           ------------------------------------------------------------\
           n";
299     cout << left << setw(30) << "List Scheduling"
300          << fixed << setprecision(4) << averageTime(listTime) << "\n
              ";
301     cout << left << setw(30) << "Sorted List Scheduling"
302          << fixed << setprecision(4) << averageTime(sortedTime) << "
              \n";
303     cout << left << setw(30) << "Local Search"
```

```
304             << fixed << setprecision(4) << averageTime(localTime) << "\
                   n";
305         cout << "
                ------------------------------------------------------------\
                n";
306
307         return 0;
308 }
```